

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

AI

Practical – VII

Roll No.: T010	Name: Sagar Kandari
Class: TYIT	Batch: 01
Date of Assignment: 1/8/26	Date/Time of Submission: 1/8/26

Machine Learning

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

- a. Load dataset using Pandas.**
- b. Perform basic preprocessing (handling missing values or scaling).**
- c. Split dataset into training and testing sets.**
- d. Train a simple classifier.**
- e. Display training and testing accuracy.**

DATASET - IRIS

```
import pandas as pd
```

[2] ✓ 0.0s Python

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

[3] ✓ 0.0s Python

```
data = pd.read_csv("iris - iris - iris - iris.csv")
```

[6] ✓ 0.0s Python

```
print("Dataset:")
print(data.head())
```

[7] ✓ 0.0s Python

Output:

```
Dataset:
  sepal_length  sepal_width  petal_length  petal_width  species
0          5.1           3.5           1.4           0.2  setosa
1          4.9           3.0           1.4           0.2  setosa
2          4.7           3.2           1.3           0.2  setosa
3          4.6           3.1           1.5           0.2  setosa
4          5.0           3.6           1.4           0.2  setosa
```

Code:

```
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

✓ 0.0s

Python

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

✓ 0.0s

Python

```
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

✓ 0.8s

Python

Output:

▼ DecisionTreeClassifier ⓘ ?

- Parameters
- Fitted attributes

Code:

```
y_pred = model.predict(X_test)
```

✓ 0.0s

Python

```
print("\nPredicted Values:")
print(y_pred)
```

✓ 0.0s

Python

Output:

```
Predicted Values:
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
```

Code:

```
print("\nActual Values:")
print(y_test.values)
```

✓ 0.0s

Python

Output:

```
Actual Values:
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
```

Code:

```
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")
```

✓ 0.0s

Python

Output:

```
Accuracy: 100.0 %
```

Code:

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

✓ 0.0s

Python

Output:

```
Classification Report:
              precision    recall  f1-score   support

   setosa               1.00      1.00      1.00        10
  versicolor            1.00      1.00      1.00         9
   virginica            1.00      1.00      1.00        11

 accuracy                   1.00         30
  macro avg              1.00      1.00      1.00         30
 weighted avg            1.00      1.00      1.00         30
```

7A-OWN DATASET BUSNIIESS.CSV

Code:

```
[16] import pandas as pd
      ✓ 0.0s Python

[17] df = pd.read_csv("BigMart_Sales.csv")
      ✓ 0.0s Python

[18] print("First 5 rows:")
      print(df.head())
      ✓ 0.0s Python
```

Output:

```
First 5 rows:
  Item_Identifier  Item_Weight  Item_Fat_Content  Item_Visibility \
0          FDA15          9.30          Low Fat          0.016047
1          DRC01          5.92          Regular          0.019278
2          FDN15         17.50          Low Fat          0.016760
3          FDX07         19.20          Regular          0.000000
4          NCD19          8.93          Low Fat          0.000000

      Item_Type  Item_MRP  Outlet_Identifier \
0          Dairy    249.8092          OUT049
1    Soft Drinks    48.2692          OUT018
2          Meat   141.6180          OUT049
3  Fruits and Vegetables  182.0950          OUT010
4      Household    53.8614          OUT013

  Outlet_Establishment_Year  Outlet_Size  Outlet_Location_Type \
0                1999      Medium      Tier 1
1                2009      Medium      Tier 3
2                1999      Medium      Tier 1
3                1998         NaN      Tier 3
4                1987       High      Tier 3

      Outlet_Type  Item_Outlet_Sales
0  Supermarket Type1          3735.1380
1  Supermarket Type2           443.4228
2  Supermarket Type1          2097.2700
3    Grocery Store           732.3800
4  Supermarket Type1           994.7052
```

Code:

```
print("\nDataset Info:")
print(df.info())
```

[19] ✓ 0.0s Python

Output:

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Item_Identifier                       8523 non-null   object
1   Item_Weight                           7060 non-null   float64
2   Item_Fat_Content                       8523 non-null   object
3   Item_Visibility                       8523 non-null   float64
4   Item_Type                             8523 non-null   object
5   Item_MRP                              8523 non-null   float64
6   Outlet_Identifier                     8523 non-null   object
7   Outlet_Establishment_Year             8523 non-null   int64
8   Outlet_Size                           6113 non-null   object
9   Outlet_Location_Type                  8523 non-null   object
10  Outlet_Type                           8523 non-null   object
11  Item_Outlet_Sales                     8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
None
```

Code:

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

[20] ✓ 0.0s Python

```
df["Item_Weight"].fillna(df["Item_Weight"].mean(), inplace=True)
df["Outlet_Size"].fillna(df["Outlet_Size"].mode()[0], inplace=True)
```

[21] ✓ 0.0s Python

Output:

C:\Users\Sagar Kandari\AppData\Local\Temp\ipykernel_28144\4253310345.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and inplace=True is specified. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object will be discarded.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'.

```
df["Item_Weight"].fillna(df["Item_Weight"].mean(), inplace=True)
```

C:\Users\Sagar Kandari\AppData\Local\Temp\ipykernel_28144\4253310345.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and inplace=True is specified. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object will be discarded.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'.

```
df["Outlet_Size"].fillna(df["Outlet_Size"].mode()[0], inplace=True)
```

Code:

```

median_sales = df["Item_Outlet_Sales"].median()
df["Sales_Class"] = (df["Item_Outlet_Sales"] >= median_sales).astype(int)
2] ✓ 0.0s Python

df.drop("Item_Outlet_Sales", axis=1, inplace=True)
3] ✓ 0.0s Python

label_encoder = LabelEncoder()
4] ✓ 0.0s Python

for col in df.select_dtypes(include="object").columns:
    df[col] = label_encoder.fit_transform(df[col])
5] ✓ 0.0s Python

X = df.drop("Sales_Class", axis=1)
y = df["Sales_Class"]
6] ✓ 0.0s Python

scaler = StandardScaler()
X = scaler.fit_transform(X)
7] ✓ 0.0s Python

from sklearn.model_selection import train_test_split
] ✓ 0.0s Python

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
] ✓ 0.0s Python

print("Training Samples:", X_train.shape[0])
print("Testing Samples :", X_test.shape[0])
] ✓ 0.0s Python

```

Output:

```

Training Samples: 6818
Testing Samples : 1705

```

Code:

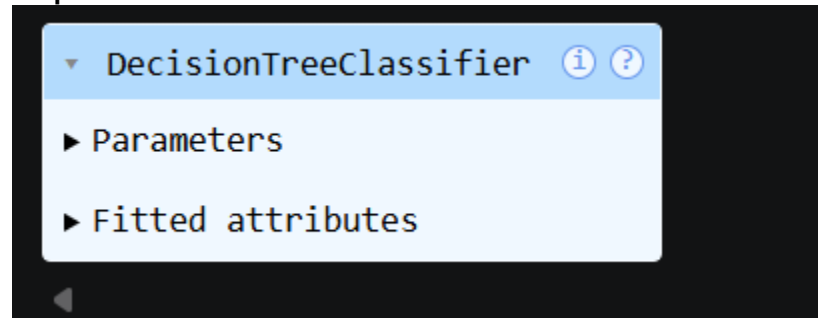
```
from sklearn.tree import DecisionTreeClassifier

classifier = DecisionTreeClassifier(random_state=42)

classifier.fit(X_train, y_train)
```

✓ 0.0s Python

Output:



Code:

```
from sklearn.metrics import accuracy_score

train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

print("Training Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy :", round(test_accuracy * 100, 2), "%")
```

✓ 0.0s Python

Output:

```
Training Accuracy : 100.0 %
Testing Accuracy  : 75.31 %
```

COMPARISON BETWEEN BOTH DATASETS

Point	Training Accuracy	Testing Accuracy
Definition	Accuracy measured on the training dataset.	Accuracy measured on the testing dataset.
Purpose	Evaluates how well the model learns the training data.	Evaluates how well the model performs on unseen data.
Dataset Used	Training dataset (X_train, y_train)	Testing dataset (X_test, y_test)
Prediction	model.predict(X_train)	model.predict(X_test)
Formula	accuracy_score(y_train, train_pred)	accuracy_score(y_test, test_pred)
Expected Value	Usually higher.	Usually slightly lower.
Indicates	Learning performance of the model.	Generalization performance of the model.
Example Result	99.20% <i>(replace with your training accuracy)</i>	87.45% <i>(replace with your testing accuracy)</i>